# Debugging, Reporting and Failure Analysis

**LESSON SPECIFICATIONS & ROADMAP**

• Track: Playwright Automation with JavaScript + AI • Level: Intermediate
• Estimated Effort: 2.5–3 Hours • Prerequisites: Days 1–9 concepts (Locators, assertions, auto-waiting, fixtures, and data-d
• Runtime: Pure Modern JavaScript (.js) • Runner: @playwright/test

Master systematic failure diagnosis in Playwright without blindly increasing timeouts. Leverage Playwright Inspector, page.pause(), Trace Viewer time-travel DOM snapshots, and enterprise HTML reporting.

## Day 10 Learning Objectives

'Categorize test failures: assertion errors, locator mismatches, timeouts, navigation issues, and app vs test defects.
'Diagnose the underlying root cause instead of blindly increasing timeouts or adding arbitrary retries.
'Use interactive debugging tools: Playwright Inspector, page.pause(), headed mode, and --debug flags.
'Investigate failed test runs in Trace Viewer: inspect action timelines, before/after DOM snapshots, and network waterfalls.
'Configure and interpret built-in HTML reports, test summaries, and CI artifacts.
'Distinguish between deterministic failures, environmental hiccups, and genuine application defects.

When an automated test fails, your job as an automation engineer is not simply to "make it green" by any means necessary. You must perform **Root Cause Analysis (RCA)**.

**The 5 Common Failure Categories**
**1. **Assertion Failures:** The application did not produce the expected text, count, or URL. (Could be an actual software bug or an outdated test assertion).**
**2. **Locator / Selection Failures:** Playwright could not find the element because the UI changed, the selector is brittle, or strict mode was violated (matched multiple elements).**
**3. **Timeouts:** Action timed out (default 30s) because an element was obscured, disabled, or an API call never resolved.**
**4. **Navigation & Network Failures:** A redirect failed, an endpoint returned a 500 server error, or CORS blocked a client request.**
**5. **Test-Data & State Dependencies:** A test failed because it relied on data modified by a preceding test that ran out of order or on another worker.**

**The Anti-Pattern: Masking Failures**
**Junior testers frequently mask failures by:**
**- Adding `await page.waitForTimeout(10000)` (arbitrary sleep).**
**- Increasing timeouts from 30s to 120s.**
**- Adding `retries: 5` in config so flaky tests eventually pass by luck.**
**- Removing or weakening assertions (`toBeTruthy()` instead of `toHaveText()`).**
**Golden Rule:** A timeout is almost always a symptom of an underlying issue, not the root cause!

**Symptom vs Root Cause (Analyzing a timeout error to uncover the true underlying bug.)**

```
// Error: Timeout 30000ms exceeded waiting for locator('button.checkout')
// 'L WRONG FIX: Increase timeout to 60000ms
// '  ROOT CAUSE INVESTIGATION:
// Looking at network tab in Trace Viewer: The /api/cart/validate returned 403 Forbidden!
// The checkout button was disabled because the cart validation failed due to an expired token.
```

Playwright provides developer-friendly debugging tools right out of the box.

**1. The Playwright Inspector**
**Run your test in debug mode:**
**```bash**
**npx playwright test --debug**
**```**

**This opens the browser in headed mode along with the **Playwright Inspector window**, allowing you to:**
**- Step through actions line-by-line (Step Over `F10`).**
**- Resume execution (`F5`).**
**- Interactively test locator expressions in the live DOM.**
**- View locator highlights and actionability status directly on the page.**

**2. Live Breakpoints with `page.pause()`**
**When debugging a specific test, place `await page.pause()` right before the failing step:**
```javascript
test('debug cart calculation', async ({ page }) => {
  await page.goto('/cart');
  await page.getByLabel('Quantity').fill('3');

  // Pauses execution and opens Inspector here!
  await page.pause();

  await expect(page.getByTestId('subtotal')).toHaveText('$90.00');
});
```

**3. Headed Execution**
**To visually watch tests execute at normal speed:**
```bash
npx playwright test --headed
```

**4. Interactive UI Mode**
**The modern Playwright visual test runner:**
```bash
npx playwright test --ui
```

**Allows instant watch-mode, live locator testing, and step-by-step DOM time-travel.**

**Targeted Debugging Command (Running a single test file in debug mode.)**

```
npx playwright test tests/checkout.spec.js -g "apply discount" --debug
```

**Trace Viewer** is Playwright's most powerful diagnostic capability. It is a post-mortem tool that captures a complete recording of your test execution.

**What Trace Viewer Captures**
**- **Action Timeline:** Visual strip showing every action with millisecond timestamps.**
**- **Before & After DOM Snapshots:** An interactive, inspectable DOM snapshot at the exact moment before and after each action.**
**- **Console Logs & Page Errors:** JavaScript errors thrown inside the browser window.**
**- **Network Waterfall:** Every HTTP request and response, including headers, status codes, and JSON response bodies.**
**- **Source Code View:** Highlights the exact line in your test script that executed.**

**How to Configure Tracing in `playwright.config.js`**
```javascript
// playwright.config.js
module.exports = {
  use: {
    // Best practice for CI: capture trace only when a test fails or retries
    trace: 'on-first-retry', // Options: 'off' | 'on' | 'retain-on-failure' | 'on-first-retry'
  },
};
```

**Opening a Trace File**
```bash
# Open locally via CLI:
npx playwright show-trace test-results/my-test-run/trace.zip
# Or load in any browser at:
# https://trace.playwright.dev (runs 100% locally in your browser!)
```

**Security & Privacy Consideration**
**Trace Viewer captures network payloads and DOM text. Avoid capturing traces on tests that display sensitive customer PII or real passwords.**

**Configuring Trace Capture for Production CI (A production playwright.config.js snippet.)**

```
const config = {
  use: {
    trace: process.env.CI ? 'retain-on-failure' : 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
};
```

Producing clear, actionable test reports ensures engineering teams can triage defects quickly.

**1. Built-in HTML Reporter**
**Playwright includes an interactive HTML reporter that aggregates pass/fail summaries, execution times, error stack traces, and attached screenshots/videos/traces:**
**```bash**
**# Configure in playwright.config.js:**
**reporter: [['html', { open: 'never' }]],**
# Open the report after a run:
npx playwright show-report
```

**2. Retries & Flaky Test Identification**
**When `retries: 2` is configured:**
**- **Passed on First Try:** Green (`passed`).**
**- **Failed First, Passed on Retry:** Orange (`flaky`).**
**- **Failed All Tries:** Red (`failed`).**
Playwright reports flaky tests separately so you can investigate race conditions before they break production deployments!

**3. Communicating Failures to Developers**
**When logging a defect found by an automated test:**
**1. Provide the exact test title and environment URL.**
**2. State the expected behavior vs actual behavior.**
**3. Attach the Playwright error message and stack trace.**
**4. Attach the trace.zip or screenshot artifact.**

**Configuring Multi-Reporters (Combining console list output with HTML file reporting in CI.)**

```
// playwright.config.js
module.exports = {
  reporter: [
    ['list'],
    ['html', { outputFolder: 'playwright-report', open: 'never' }],
  ],
};
```

requirements.
```

### Assignment Starter Code (Day 10)

```
// tests/diagnose_and_fix_suite.spec.js
// & þ  LAB CHALLENGE: This test suite contains 6 deliberate automation defects.
// Your task is to diagnose each failure, fix the code, and explain the root causes!

const { test, expect } = require('@playwright/test');

test.describe('E-Commerce Order Management - Diagnostic Lab', () => {

  // DEFECT 1: Incorrect locator strategy (brittle CSS class that changed in redesign)
  // DEFECT 2: Test data dependency (hardcoded email that collides)
  test('TC-1: Customer account registration and dashboard access', async ({ page }) => {
    // FIX: Use unique collision-free email
    const uniqueEmail = `customer_${Date.now()}@apexcorp.io`;

    await page.goto('/signup');
    await expect(page.getByRole('heading', { name: /create account|sign up/i })).toBeVisible();

    // Filling form fields
    await page.getByLabel(/full name/i).fill('Meera Patel');
    await page.getByLabel(/email/i).fill(uniqueEmail);
    await page.getByLabel(/password/i).fill('ValidPassword@2026');
```

## DAY 10 INTERVIEW PREPARATION QUESTION BANK

### Q1. How do you investigate a failing Playwright test when you only have access to CI logs?

**[Difficulty: Intermediate] Evaluation Focus: CI failure triage and artifact analysis.**

Model Answer: First, I inspect the failure stack trace in the CI log to identify the failing file, line number, and error type (assertion failure, locator timeout, or network error). Next, I download the CI artifacts—specifically the Playwright HTML report and trace.zip file. I open the trace locally using `npx playwright show-trace trace.zip` to inspect the DOM snapshots, console errors, and network request waterfall at the exact moment of failure. This pinpoints whether the defect was caused by an application bug (e.g. 500 error), an environment issue, or a test synchronization bug.

*Follow-Up Question: "How does trace: "retain-on-failure" help optimize CI storage while aiding debugging?"*

### Q2. What is the difference between Playwright Inspector and Trace Viewer?

**[Difficulty: Intermediate] Evaluation Focus: Tooling distinctions and appropriate usage.**

Model Answer: Playwright Inspector is a live, interactive debugging tool used during local development (via `--debug` or `page.pause()`) to pause execution and step through actions line-by-line in real-time. In contrast, Trace Viewer is a post-mortem time-travel analysis tool that plays back a pre-recorded test run. Trace Viewer is ideal for investigating failures that occurred in headless CI environments where no live human interaction was possible.

*Follow-Up Question: "Can Trace Viewer be viewed in a browser without installing Playwright locally?"*

### Q3. Why is adding arbitrary sleep timers (page.waitForTimeout) considered an anti-pattern for fixing test failures?

**[Difficulty: Beginner] Evaluation Focus: Anti-flakiness engineering and web-first waiting.**

Model Answer: Adding fixed sleep timers masks the symptom rather than solving the root cause. If network latency fluctuates beyond the sleep duration, the test fails anyway. Furthermore, sleep timers artificially inflate suite execution time across hundreds of tests. The professional solution is to identify the condition being waited on (e.g. element visibility, URL change, or API response) and use web-first auto-retrying assertions or `page.waitForResponse()` instead.

*Follow-Up Question: "What is the difference between actionability auto-waiting and assertion auto-waiting?"*

### Q4. How do retries work in Playwright, and when is it appropriate to configure them?

**[Difficulty: Intermediate] Evaluation Focus: Test stability, CI configuration, and flaky test management.**

Model Answer: Retries are configured in `playwright.config.js` via the `retries` option (typically 1 or 2 in CI). When a test fails, Playwright destroys the browser context and re-runs the test in a clean environment. If it passes on retry, Playwright flags it as "flaky" in reports rather than "passed." Retries prevent temporary network blips from blocking CI deployments, but flaky tests must be prioritized and fixed, not ignored.

*Follow-Up Question: "Why should retries typically be set to 0 during local development?"*

### Q5. How do you differentiate between an application defect and an automation script defect during triage?

**[Difficulty: Scenario-Based] Evaluation Focus: Critical thinking and quality assurance leadership.**

Model Answer: I reproduce the scenario manually in the target environment using the exact test data. If the application returns an unhandled error, displays incorrect data, or fails to navigate, it is an application defect. I check the network tab in the trace: if an API returned 500 Internal Server Error or an invalid payload, the test accurately caught a software defect. If the application functions correctly manually but the test failed due to an outdated selector, a strictness violation, or a premature non-retrying assertion, it is an automation script defect.

*Follow-Up Question: "What steps do you take when an application defect is confirmed by automation?"*

### 1. Which CLI flag launches Playwright tests in interactive step-through debugging mode with the Inspector?

Key Concept: npx playwright test --debug opens the Playwright Inspector GUI and launches browsers in headed mode for step-by-step interactive debugging.

### 2. What is the function of await page.pause() in a Playwright test script?

Key Concept: page.pause() acts as a programmatic breakpoint, halting execution and opening the Playwright Inspector so you can inspect locators and DOM state.

3. **What does Trace Viewer allow engineers to inspect that static screenshots cannot provide?**
Key Concept: Trace Viewer records interactive time-travel DOM snapshots, network waterfalls, console messages, and source lines for every executed step.

4. **Which trace configuration is recommended for CI to save storage while capturing evidence for failing tests?**
Key Concept: 'retain-on-failure' discards trace files for passing tests and only preserves traces when a test fails, optimizing CI storage and upload bandwidth.

5. **How can you view a Playwright trace.zip file in a web browser without running local commands?**
Key Concept: trace.playwright.dev is a client-side Progressive Web App that runs entirely in your browser without uploading your trace data to external servers.

6. **What command opens the built-in Playwright HTML report after a test run finishes?**
Key Concept: npx playwright show-report starts a local web server and serves the interactive HTML report generated from the latest test execution.

7. **Why should automation engineers avoid increasing timeouts from 30s to 120s when a test times out?**
Key Concept: Blindly extending timeouts masks underlying problems like deadlocked network requests or missing elements, multiplying CI build times without improving reliability.

8. **What is a "strict mode violation" error in Playwright?**
Key Concept: Playwright enforces strict mode: if a locator matches 2 or more elements, action calls like click() or fill() throw a strict mode violation to prevent clicking the wrong element.

9. **What is the modern command to launch Playwright's interactive visual UI Mode?**
Key Concept: npx playwright test --ui opens Playwright's interactive UI mode featuring live watch mode, visual time-travel, and instant locator picking.

10. **What is the primary indicator of a genuine application defect when analyzing a failed automated test in Trace Viewer?**
Key Concept: When Trace Viewer shows server 500 errors, unhandled JavaScript runtime exceptions, or missing data from the backend, the automated test has successfully caught a real application bug.